COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE031	**SECTION:**	CPE21S2
DATE PERFORMED:	01-08-2026	**DATE SUBMITTED:**	01-08-2026
NAME:	GUARIN, JAN MIKAELA C.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 2 Interactive Widget

1. PROCEDURE OUTPUT

ILO A.

Screenshot

```cpp
#include <iostream>
#include <stack>

int main() {
    std::stack<int> stack1;
    std::cout << "====Testing the stack STL=====" << std::endl;
    std::cout << "\n--------- STACK 1 ---------" << std::endl;

    //isEmpty
    std::cout << "is the stack empty? " << stack1.empty() << std::endl;

    //push
    stack1.push(10);
    std::cout << "The top of the stack is:  " << stack1.top() << std::endl;
    stack1.push(9);
    std::cout << "The top of the stack is:  " << stack1.top() << std::endl;
    stack1.push(8);
    std::cout << "The top of the stack is:  " << stack1.top() << std::endl;
    stack1.push(7);
    std::cout << "The top of the stack is:  " << stack1.top() << std::endl;
    stack1.pop();
    std::cout << "The top of the stack is:  " << stack1.top() << std::endl;

    std::cout << "The size of the stack is: " << stack1.size() << std::endl;

    //emplace
    std::stack<int> stack2;
    std::cout << "\n--------- STACK 2 ---------" << std::endl;

    stack2.emplace(10);
    std::cout << "The top of the stack 2 is: " << stack2.top() << std::endl;

    stack2.emplace(15);
    std::cout << "The top of the stack 2 is: " << stack2.top() << std::endl;

    stack2.emplace(22);
    std::cout << "The top of the stack 2 is: " << stack2.top() << std::endl;

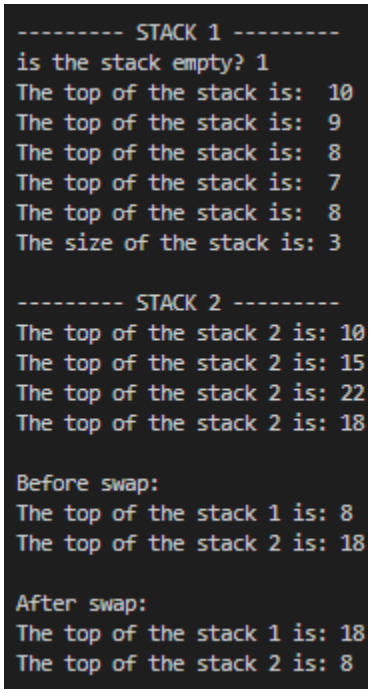
    stack2.emplace(18);
    std::cout << "The top of the stack 2 is: " << stack2.top() << std::endl;

    //swapping

    std::cout << "\nBefore swap:" << std::endl;
    std::cout << "The top of the stack 1 is: " << stack1.top() << std::endl;
    std::cout << "The top of the stack 2 is: " << stack2.top() << std::endl;

    stack1.swap(stack2);

    std::cout << "\nAfter swap:" << std::endl;
    std::cout << "The top of the stack 1 is: " << stack1.top() << std::endl;
    std::cout << "The top of the stack 2 is: " << stack2.top() << std::endl;
}
```

Output	
Analysis	This program demonstrates how the C++ Standard Template Library (STL) can be used to perform stack operations without implementing the data structure from scratch. It utilizes built-in functions such as **push()**, **pop()**, **top()**, **empty()**, **size()**, **emplace()**, and **swap()** to manipulate and manage stack elements efficiently. The program also compares the contents of two stacks before and after performing a swap operation, illustrating how entire stacks can be exchanged with a single function call.

ILO B1.

Screenshot

```cpp
#include <iostream>

// global declarations
#define maxCap 10

int stackArr[maxCap];
int top = -1, newData;

// prototype functions
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void display();

int main() {
    // main driver
    int choice;

    while (true) {
        std::cout << "\n====================" << std::endl;
        std::cout << "   Stack operations  " << std::endl;
        std::cout << "====================" << std::endl;
        std::cout << " 1. Push" << std::endl;
        std::cout << " 2. Pop" << std::endl;
        std::cout << " 3. Top" << std::endl;
        std::cout << " 4. isEmpty" << std::endl;
        std::cout << " 5. isFull" << std::endl;
        std::cout << " 6. Display" << std::endl; // New menu option added
        std::cout << "====================" << std::endl;
        std::cout << "Enter choice: ";
        std::cin >> choice;

        std::cout << "--------------------" << std::endl;

        switch(choice) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                Top();
```

```
            case 4:
                std::cout << "is stack empty? " << (isEmpty() ? "True" : "False") << std::endl;
                break;
            case 5:
                std::cout << "is the stack full? " << (isFull() ? "True" : "False") << std::endl;
                break;
            case 6:
                display(); // New case added
                break;
            default:
                std::cout << "Invalid choice." << std::endl;
                break;
        }
    }

    return 0;
}

// function definition

bool isEmpty() {
    // how do we verify if the stack is empty
    if (top == -1) return true;
    return false;
}

bool isFull() {
    // how do we verify if stack is full?
    if (top == maxCap - 1) return true;
    return false;
}

void push() {
    // error checking
    if (isFull()) {
        std::cout << "Stack overflow" << std::endl;
        return;
    }

    // pushing to the stack
    std::cout << "Enter a new Value: ";
    std::cin >> newData;
```

```
    std::cout << "Enter a new Value: ";
    std::cin >> newData;

    // how do we insert the data into the stack?
    stackArr[++top] = newData;
    std::cout << newData << " pushed to stack." << std::endl;
}

void pop() {
    // error checking
    if (isEmpty()) {
        std::cout << "Stack underflow" << std::endl;
        return;
    }

    // Display the value that we are going to pop:
    std::cout << "Popping: " << stackArr[top] << std::endl;

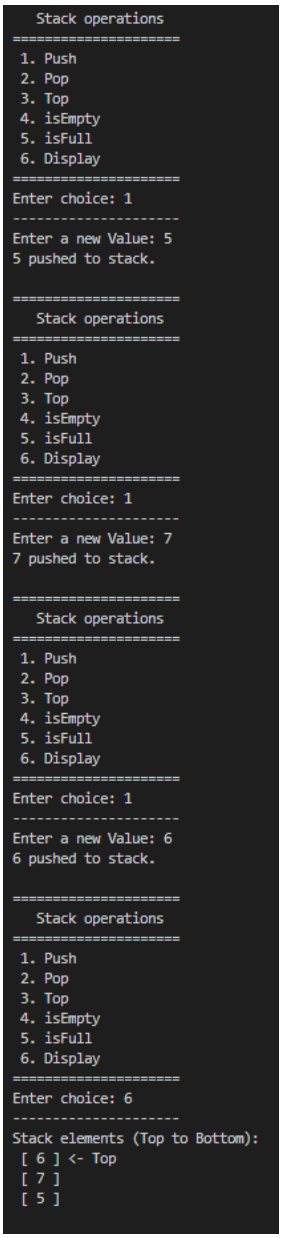
    // Decrement the top value from the stack
    top--;
}

void Top() {
    // error catching:
    if (isEmpty()) {
        std::cout << "The stack is empty" << std::endl;
        return;
    }

    // check the top value
    std::cout << "top element: " << stackArr[top] << std::endl;
}

// display function
void display() {
    if (isEmpty()) {
        std::cout << "The stack is empty, nothing to display." << std::endl;
        return;
    }

    std::cout << "Stack elements (Top to Bottom):" << std::endl;
    for (int i = top; i >= 0; i--) {
        std::cout << " [ " << stackArr[i] << " ] ";
        if (i == top) {
```

Output	
Analysis	This program demonstrates the implementation of a stack using an array with a fixed maximum capacity. It provides the basic stack operations, including push, pop, Top, isEmpty, isFull, and display, allowing users to interact with the stack through a menu-driven interface. The program also handles overflow and underflow conditions to prevent invalid operations when the stack is full or empty. It also showed the limitation of an array-based stack, where the maximum number of elements is restricted by its predefined size.

Screenshot

```cpp
#include <iostream>
#include "stackList.h"

int main()
{

    std::cout << "--- PUSHING 'C' 'P' 'E' ONTO THE STACK ---\n";

    // PUSHING
    push('C');
    push('P');
    push('E');

    // DISPLAY THE CURRENT TOP

    Top<char>; /* TOP ELEMENT SHOULD BE 'E' */
    displayALL<char>();
    std::cout << "\n --- POPPIING ALL ELEMENT ---\n";

    while (!isEmpty<char>())
    {
        std::cout << "Popped: " << pop<char>() << std::endl;
    }

    std::cout << "\n --- TESTING THE UNDERFLOW CONDITION --- \n";
    pop<char>(); /* UNDERFLOW CONDITION */

    return 0;
}
```

main.cpp

```cpp
#ifndef STACKLIST_H
#define STACKLIST_H

#include <iostream>

template <typename T>
class Node
{
public:
    T data;     // VALUE WE NEED TO STORE
    Node *next; // POINTER TO THE NEXT NODE
};

// GLOBAL POINTER FOR THE TOP OF THE STACK
template <typename T>
Node<T> *head = nullptr;

// OPERATTIONS FOR THE STACK

// PUSH
template <typename T>
void push(T newData)
{
    // CREATE A NEW NODE AND ASSIGN THE newData
    Node<T> *newNode = new Node<T>();
    newNode->data = newData;

    // NEW NODE POINTS TO CURRENT TOP, THEN BECOMES THE NEW TOP
    newNode->next = head<T>;
    head<T> = newNode;
}

// POP

template <typename T>
T pop()
{
    // CHECK IF THE STACK IS EMPTY:
    if (head<T> == nullptr)
    {
        std::cerr << "Stack Underflow: Cannot pop from an empty stack." << std::endl;
    }

    // SAVE THE CURRENT TOP NODE
    Node<T> *temp = head<T>;
    // STORE ITS DATA
    T tempVal = temp->data;

    // MOVE THE TOP POINTER TO THE NEXT NODE
    head<T> = head<T>->next;

    // FREE THE REMOVED NODE
    delete temp;
```

```
    // RETURN THE REMOVED VALUE
    return tempVal;
}

// isEmpty
template <typename T>
bool isEmpty()
{
    return head<T> == nullptr;
}

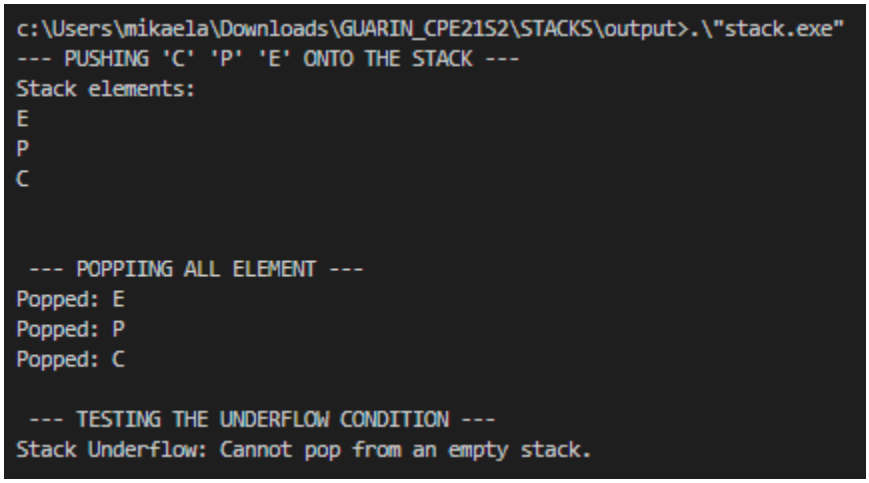
// Top
template <typename T>
void Top()
{
    // ERROR CATCHING:
    if (head<T> == nullptr)
    {
        std::cout << "Stack is empty. No top element." << std::endl;
        return;
    }
    // DISPLAY THE TOP ELEMENT
    std::cout << "Top element: " << head<T>->data << std::endl;
}

// DISPLAY ALL
template <typename T>
void displayALL()
{
    // ERROR CATCHING:
    if (head<T> == nullptr)
    {
        std::cout << "Stack Underflow\n"
                  << std::endl;
        return;
    }

    // TRAVERSE THE STACK AND DISPLAY ALL ELEMENTS
    Node<T> *temp = head<T>;
    std::cout << "Stack elements: \n";
    while (temp != nullptr)
    {
        std::cout << temp->data << " \n";
        temp = temp->next;
    }
    std::cout << std::endl;
}

#endif // STACKLIST_H
```

stackList.h

Output	
Analysis	This program allowed me to explore the core operations of a stack by implementing it with a linked list. Each operation, including push, pop, Top, and displayALL, has a specific role in managing the data stored in the stack. Performing these operations showed how the most recently added element is always the first one to be removed, following the Last-In, First-Out (LIFO) principle.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C.

Screenshot

stackList.h 1 | balancecheck.cpp | stack.cpp | Guarin_Seatwork4_1_main_ILC

stackList.h > isMatching(char, char)

```
#ifndef STACKLIST_H
void displayALL()
Click to add a breakpoint
// SUPPLEMENTARY

bool isMatching(char open, char close)
{
    if (open == '(' && close == ')')
        return true;

    if (open == '[' && close == ']')
        return true;

    if (open == '{' && close == '}')
        return true;

    return false;
}

bool isBalanced(std::string expression)
{
    while (!isEmpty<char>())
    {
        pop<char>();
    }

    for (int i = 0; i < expression.length(); i++)
    {
        char current = expression[i];

        if (current == '(' || current == '[' || current == '{')
        {
            push<char>(current);
        }
        else if (current == ')' || current == ']' || current == '}')
        {
            if (isEmpty<char>())
            {
                return false;
            }

            char open = pop<char>();

            if (!isMatching(open, current))
            {
                return false;
            }
        }
    }

    if (!isEmpty<char>())
    {
        return false;
    }

    return true;
}

#endif // STACKLIST_H
```

```cpp
#include <iostream>
#include <string>
#include "stackList.h"

using namespace std;

int main()
{
    string expression;
    char answer;

    do
    {
        cout << "\n==============================" << endl;
        cout << "   BALANCED SYMBOL CHECKER" << endl;
        cout << "==============================" << endl;

        cout << "Input an expression: ";
        getline(cin, expression);

        bool result = isBalanced(expression);

        if (result)
        {
            cout << "\nThe expression is BALANCED." << endl;
        }
        else
        {
            cout << "\nThe expression is NOT BALANCED." << endl;
        }

        cout << "\nTry another expression? (Y/N): ";
        cin >> answer;
        cin.ignore(1000, '\n');

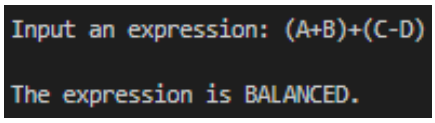
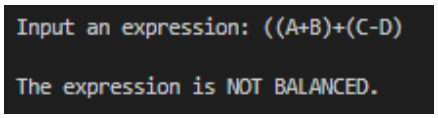
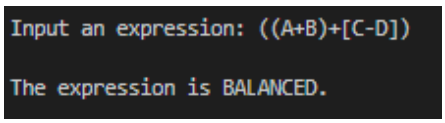
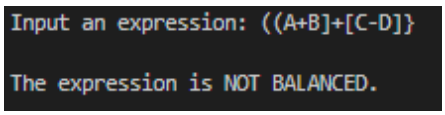
    } while (answer == 'Y' || answer == 'y');

    cout << "\nThank you for using the program!" << endl;

    return 0;
}
```

balancecheck.cpp

Output	``` ========== BALANCED SYMBOL CHECKER ========== Input an expression: (A+B)+(C-D) The expression is BALANCED. Try another expression? (Y/N): y ========== BALANCED SYMBOL CHECKER ========== Input an expression: ((A+B)+(C-D) The expression is NOT BALANCED. Try another expression? (Y/N): y ========== BALANCED SYMBOL CHECKER ========== Input an expression: ((A+B)+[C-D]) The expression is BALANCED. Try another expression? (Y/N): y ========== BALANCED SYMBOL CHECKER ========== Input an expression: ((A+B]+[C-D]} The expression is NOT BALANCED. Try another expression? (Y/N): n Thank you for using the program! ```
Analysis	This program demonstrated how stack operations can be used to determine whether an expression contains balanced symbols. It processes each character one at a time, pushing every opening parenthesis, bracket, or brace onto the stack and popping it when the corresponding closing symbol is encountered. If the symbols do not match or if there are extra opening or closing symbols, the expression is identified as unbalanced.

Expression	Valid? (Y/N)	Output(Console Screenshot)	Analysis
(A+B)+(C-D)	Y		Every opening symbol has the correct closing symbol, making the expression balanced.
((A+B)+(C-D)	N		One opening parenthesis is left without a matching closing parenthesis.
((A+B)+[C-D])	Y		Both parentheses and brackets are correctly paired and properly nested.
((A+B]+[C-D]}	N		Some opening symbols are closed with the wrong symbol, so the expression is not balanced.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.